

B.Sc. in Computer Science and Engineering

FedCrime: A Federated Learning Based Crime Prediction Model

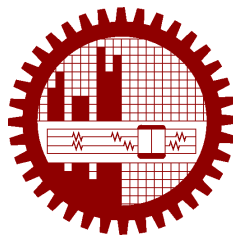
Submitted by

Prangon Chakraborty
1805085

Md. Farhan Mahtab
1805096

Supervised by

Dr. Tanzima Hashem



**Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology**

Dhaka, Bangladesh

June 2024

CANDIDATES' DECLARATION

This is to certify that the work presented in this thesis, titled, "FedCrime: A Federated Learning Based Crime Prediction Model", is the outcome of the investigation and research carried out by us under the supervision of Dr. Tanzima Hashem.

It is also declared that neither this thesis nor any part thereof has been submitted anywhere else for the award of any degree, diploma or other qualifications.

Prangon Chakraborty
1805085

Md. Farhan Mahtab
1805096

CERTIFICATION

This thesis titled, ”**FedCrime: A Federated Learning Based Crime Prediction Model**”, submitted by the group as mentioned below has been accepted as satisfactory in partial fulfillment of the requirements for the degree B.Sc. in Computer Science and Engineering in June 2024.

Group Members:

Prangon Chakraborty

Md. Farhan Mahtab

Supervisor:

Dr. Tanzima Hashem

Professor

Department of Computer Science and Engineering

Bangladesh University of Engineering and Technology

ACKNOWLEDGEMENT

We thankfully show our utmost gratitude and honor to our undergraduate thesis supervisor (Dr. Tanzima Hashem, Professor, BUET) Madam for her constant support, guidance and motivation throughout our thesis.

We would like to show our gratitude to (Sukarna Barua, Associate Professor, BUET) Sir for his continuous support and guidance.

We thank the Research and Innovation Centre For Science and Engineering, RISE-BUET for funding this thesis project under the RISE Undergraduate Research Grant S2023-03-051.

Dhaka
June 2024

Prangon Chakraborty
Md. Farhan Mahtab

Contents

<i>CANDIDATES' DECLARATION</i>	i
<i>CERTIFICATION</i>	ii
<i>ACKNOWLEDGEMENT</i>	iii
List of Figures	vi
List of Tables	vii
	viii
<i>ABSTRACT</i>	ix
1 Introduction	1
1.1 Problem Setup	3
1.2 Solution Overview	4
1.3 Contribution	6
1.4 Organization	6
2 Related Works	7
2.1 Crime Prediction Models	7
2.1.1 Statistical and Machine Learning Models	7
2.1.2 Deep Learning Models	8
2.2 Federated Learning	8
2.2.1 Data Partitioning	9
2.2.2 Aggregation Functions	9
2.2.3 Clustering	11
2.3 Summary	11
3 Our Model	14
3.1 System Overview	14
3.2 Clustering Method	16
3.3 Client and Server Side Components	17

3.4	Weight Sharing Method	17
3.5	Data Sharing Method	19
3.5.1	Server Side Data Partitioning	20
3.5.2	Server Data Sharing with Cluster Heads	20
3.5.3	Model Training and Weights Aggregation	21
3.5.4	Model Architecture	22
3.5.5	Aggregation Algorithms	24
4	Experiments	28
4.1	Experiment Setup	28
4.1.1	Datasets	28
4.1.2	Methods	29
4.1.3	Hyperparameters	29
4.1.4	Performance Measures	29
4.2	Results and Discussions	29
4.2.1	Comparative Assessment	30
4.2.2	Effect of Varying Cluster Size	30
4.2.3	Effect of Varying Server Dataset Size	32
5	Conclusion and Future Work	34
	References	36

List of Figures

3.1	System Overview of FedCrime	15
3.2	Clustering Method in Federated Learning	16
3.3	Weight Sharing Method	18
3.4	Server Side Data Partitioning	20
3.5	Server Data Sharing With Cluster Heads	21
3.6	Model Training and Weights Aggregation	22
3.7	Model Architecture of FedCrime	23
4.1	Effect of varying cluster size on MAE	31
4.2	Effect of varying cluster size on MSE	31
4.3	Effect of varying Server Datasize on MAE	32
4.4	Effect of varying Server Datasize on MSE	33

List of Tables

4.1 Comparison of Different Federated Learning Frameworks	30
---	----

List of Algorithms

1	FedCrime with FedAvg	24
2	FedCrime with FedAvg Weighted Aggregation	25
3	FedCrime with FedProx	26
4	FedCrime with FedProx Weighted Aggregation	27

ABSTRACT

Crime prediction is crucial for crime prevention. To improve crime prediction accuracy, recent research has focused on developing deep-learning architectures that can learn from historical crime data. However, unreported incidents due to privacy concerns and delayed reporting lead to incomplete and outdated public crime data, affecting the accuracy of current prediction methods. To enhance prediction accuracy, capturing information on maximum crime incidents is essential and at the same time, users' privacy needs to be preserved. To address this issue, we introduce FedCrime, a novel crime prediction model using federated learning. Traditional federated learning [1] framework trains models on clients' private data. The user models are then aggregated on a central server to form a global model for prediction without transferring client data, thus preserving privacy. However, training crime prediction models on individual user data cannot produce a model with good prediction accuracy due to the very less crime incident data available from most users.

To handle the data sparsity challenge, we adopt the *clustering method*, where users form clusters with their trusted peers. Each cluster head merges its own data with those from the devices of other users in the cluster. The cluster heads participate in federated learning. We first propose clustering-based *weight sharing method*, where the server model is trained with public crime data before sending the model to each cluster head for further training. Each cluster head initializes its model with the server model and individually trains on its private data. After training, the models are sent to the server and the server aggregates the model. These steps are repeated for some rounds and the final model is used for crime prediction. To further reduce the data sparseness problem, we propose our novel clustering-based *data sharing method*, where the public crime dataset from the server can be merged with the user's private data. Specifically, before starting the training, the server divides the samples created from the public crime dataset into an equal number of partitions as the number of cluster heads and sends to the cluster heads. The cluster heads aggregate their data with the received data and the general federated learning process continues.

Extensive experiments on various federated learning settings with real datasets in the simulated environment show that our novel *data sharing method* outperforms the *weight sharing method* with clustering. Furthermore, we modify a popular FedProx aggregation method and our results show that when clients have much more data than the server, our novel aggregation method outperforms existing literature.

Chapter 1

Introduction

Crime prediction allows local authorities to take preemptive measures to prevent crime incidents. Knowing about the probable crime events, travellers can take the necessary precautions and plan a safe route before visiting a place. The prediction of crime incidents can help to significantly reduce crime rate and improve the security sector of a country.

Various crime prediction models have been proposed in recent decades. Crime prediction models can be categorized into two groups: "Statistical and Classic ML Based Models" and "Deep Learning Based Models". In "Statistical and Classic ML Based Models", various Regression Methods [2] [3] and Random Forest [4], Kernel Density Estimation (KDE) [5,6] have been utilized to capture spatial crime patterns to predict crime occurrence. To address the temporal dynamics, time-series models such as Autoregressive Integrated Moving Averages (ARIMA) [7] have been proposed to predict crimes. Some recent studies [8,9] leverage both spatial and temporal correlation of historical crime data.

Deep Learning Models like HAGEN [10] utilized an adaptive graph learning module along with DCGRU for crime prediction. Variants of GAT and SAB-LSTM are used by AIST [11] to predict crime. ST-SHL [12] captures the spatio-temporal crime pattern with a Multiview Spatial-Temporal Convolutional Network and integrates hypergraph structure learning by the graph neural network. A hierarchical spatial dependency encoder and a graph temporal shift mechanism are used by ST-SHN [13] to predict crime incidents.

All the above models use centralized public crime datasets. However, the major concern with using centralized datasets in crime prediction models is that centralized datasets are not complete and always up-to-date. Though technological advancement has made data capture and storage much easier, people do not feel comfortable reporting the crime incidents that they face for privacy reasons. For example, women would not like to disclose the harassment that they encounter on the roads. People who get life threats may not disclose the incidents they face for similar reasons. Many crime incidents are happening every day but not all of them are being

reported. A UK-based crime rate study¹ shows that 6 out of 10 crime incidents go unreported by the victims. Only 40 percent of crime incidents are recorded in the public crime dataset. Moreover, people often delay their crime reporting. Therefore, a huge portion of crime incidents remain hidden and the crime prediction models may not capture the actual state, and prediction accuracy will be poor.

To resolve this issue, crowdsourced data can become helpful. If people can contribute to the crime prediction model without sacrificing privacy of their crime incidents, the impact of the lack of data in the centralized datasets on the crime prediction model can be reduced. In this thesis, we address the challenge and develop a federated learning-based crime prediction model, FedCrime, where people do not need to share their crime data with a centralized server and thus the relevant privacy issue will be resolved. Our objective is to use Federated Learning to ensure user data privacy and use the crowdsourced data to improve the accuracy of crime prediction models.

In recent years, researchers have proposed a variant of federated learning architectures to allow data-driven solutions for challenges posed by privacy concerns and distributed data sources. For example, the federated learning approach has improved the accuracy of the breast cancer detection model [14], code smell detection model [15], intrusion detection for transportation IoT [16] etc. In Federated Learning, the user's private crime data is used in training machine learning models without violating the user's privacy. This can be done by using the trained model from the user instead of directly using the user's private data. The traditional federated learning method randomly initializes a crime prediction model and sends the model to all users for parallel training. Then the users train their models using their data and send the trained models to the server. The server aggregates the model parameters using an aggregation algorithm and sends the aggregated model back to the users for the next round of training and the training continues for a certain number of rounds. The server stores the final trained model for prediction or classification depending on the nature of the problem.

However, this traditional federated learning framework is not suitable in our context. As most of the user contains a minimal amount of crime incident data, training a crime prediction model solely on user data cannot produce a model with good prediction accuracy. In our crime prediction model, we also have public crime data collected from various public institutions. However, we need to incorporate this significant amount of data alongside the user's private data in our federated learning framework to reduce the data sparsity problem further and enhance prediction accuracy.

Therefore, we introduce *clustering method* in our federated learning framework. *Clustering method* introduces a hierarchical structure that enhances both the efficiency and effectiveness of data processing and model training. The process begins by dividing all participating clients into

¹<https://crimerate.co.uk/>

a certain number of clusters. This division is based on trusted circle criteria where in a trusted circle, all users trust each other. For instance, clients located in the same metropolitan area who trust each other might be grouped together due to their shared urban crime patterns, whereas rural clients might form a separate cluster reflecting their unique crime challenges. Within each cluster, one client is selected as the cluster head. The cluster head acts as the primary point of communication for the cluster, consolidating the data and updates from all member clients.

Only the cluster heads participate in the federated learning process with the central server. They send the aggregated model updates to the server, which in turn, aggregates these inputs from all cluster heads to update the global model. The updated global model is then sent back to the cluster heads for the next rounds of training. After a certain number of rounds, we get the final model for crime prediction. We call this method *weight sharing method*.

However, even after clustering, the data sparsity could not be reduced significantly. This is because most of the users have very few data and maximum time-steps have zero crime incidents for the users. So after clustering by maintaining time-step consistency, maximum time-steps have zero (0) crime incidents and our samples are still sparse. If we can integrate the public crime dataset that is available in the server and merge them with the cluster heads' data, then the data sparsity will be reduced further and it will not conflict with our privacy issue.

Therefore, we introduce our novel *data sharing method* where we utilize the clustering method and our novel data sharing approach to handle the data sparsity problem further. In *data sharing method*, server data is divided the server into an equal number of partitions as the total number of cluster heads and then each partition of the server data is merged with each cluster head maintaining the time-step consistency. There is no additional server-side model training. The crime prediction model is first randomly initialized with model parameters in the server and sent to all the cluster heads. The cluster heads train the model with their merged data and send the models to the server and the server aggregates all the models and sends back the aggregated model to the cluster heads for further training rounds. This training runs for a certain number of rounds and we get the final crime prediction model. Our novel *data sharing method* outperforms the *weight sharing method*. We have introduced a novel *weighted aggregation method* named *FedProx with Weighted Aggregation* and utilized it in our FedCrime architecture.

1.1 Problem Setup

In this study, we assume that the city is divided into administratively defined N geographic regions where each region fulfills statistical and planning purposes. Let $R \in \mathbb{R}^N$ represents the set of N regions $\{r_1, r_2, r_3, \dots, r_N\}$ where r_i represents the i th region. We also assume that there are K types of crime categories (*i.e.* Theft, Battery, Narcotics) and T consecutive historical time slots (*i.e.* hours).

We assume that our federated learning system has a server S containing a public historical crime dataset χ . Let $x_{i,t}^k \in \mathbb{R}$ be the number of crime occurrences of category k at region r_i during t -th time step. If $\mathbf{x}_{i,t} = [x_{i,t}^1, x_{i,t}^2, x_{i,t}^3, \dots, x_{i,t}^K] \in \mathbb{R}^K$ denotes the reported crimes of all K categories at region r_i during t -th time step and $\mathbf{X}_t = (\mathbf{x}_{1,t}, \mathbf{x}_{2,t}, \dots, \mathbf{x}_{N,t}) \in \mathbb{R}^{K \times N}$ denotes the reported crimes of all categories at all N regions during t -th time step, then the crime occurrences of the whole city for T time steps can be denoted as $\chi = (\mathbf{X}_1, \mathbf{X}_2, \mathbf{X}_3, \dots, \mathbf{X}_T) \in \mathbb{R}^{N \times K \times T}$.

We also assume that our federated learning system has C clients indicated by $\{u_1, u_2, \dots, u_C\}$. We assume that all the C clients have private training data $\delta = \{D_1, D_2, \dots, D_C\} \in \mathbb{R}^C$ Where $D_j \in \mathbb{R}^{N \times K \times T}$ represents private crime occurrence data of u_j client.

Here, $D_j = (d_1(j), d_2(j), \dots, d_T(j))$ where $d_t(j) \in \mathbb{R}^{K \times N}$ represents crimes of all categories K and in all regions N faced by the u_j client during t th time step.

Here, $d_t(j) = (\Delta_{1,t}(j), \Delta_{2,t}(j), \dots, \Delta_{N,t}(j))$ where $\Delta_i \in \mathbb{R}^K$ represents all crimes of all categories in r th region faced by u_c client during t th time step.

Finally, $\Delta_{i,t}(j) = (\psi_{i,t}^1(j), \psi_{i,t}^2(j), \dots, \psi_{i,t}^K(j))$ where $\psi_{i,t}^k(j) \in \mathbb{R}$ represent the total number of crimes of k th crime category in r_i region at t th time step faced by u_j client.

Problem Definition. Given past public crime occurrences χ and private crime occurrences δ of C clients for the last T time steps, predict $\hat{Y}_{T+1}$, the crime occurrences of the city during $(T + 1)$ th time step.

1.2 Solution Overview

To solve our problem, we first implement a basic federated learning setting. In the traditional federated learning setting, there are several user devices that we are defining as clients. Each client must have its own private data. For our experiment, we only have the public crime dataset of Chicago for 77 regions. So we first divide the crime dataset into two portions. We define the first portion as the server dataset that is public and will be stored in the server. The second portion is the private dataset that will be distributed among a certain number of client devices. Thus, we are simulating the federated learning environment where we have client devices with their private data and a server with server data. Then according to the traditional federated learning settings, the server model is first initialized with random model parameters. The initial model is sent to each client device. We trained the initial crime prediction model in each user's device. After training, each model is sent to the server and the server aggregates the results (weights/parameters) of each model and reinitializes the server model with the aggregated result. Furthermore, we refine the result by training the reinitialized server with the publicly available crime dataset stored in the server. Then the newly trained server model is sent back to all the client devices for the next round of training. This training process is executed for

several rounds. However, we couldn't find improvement in results because the user data was not sufficient enough. There was a huge sparsity in user crime data. In the user data samples, a maximum number of time stamps have 0 crime incidents. It is logical because a user may not face huge number of crimes in a single timestamp. Therefore, the crime prediction model was getting zero crime incidents in the training process.

On the other side, the central dataset has a certain number of crime incidents in a single timestamp. There is a temporal pattern in the server data. Therefore, no improvement was found in the user-side model training. Rather the model was refined mainly on the central dataset. That's why we couldn't find any improvement in the federated learning training. We need to improve the user data and reduce sparsity so that the crime prediction model on the user side can capture the pattern.

To resolve this issue, we adapt the concept of *clustering* [17]. Firstly, we divide the users into a certain number of clusters. Each cluster forms a trusted circle that has a cluster head user and all the users inside that cluster head trust the cluster head. Therefore, all the users inside a particular cluster share their data with the cluster head and the cluster head has all the data of other users of that cluster, merged with its data. Only the cluster heads participate in the federated training. In this way, we try to reduce the sparsity of the data of the cluster heads. Afterward, instead of having a randomly initialized server model, we first train the initial server model with the public dataset available on the server. Then this model is sent to all the cluster heads and each cluster head trains the model with their data. Afterward, the cluster heads send back their models to the server, the server aggregates the models and further refines them by training with the public data. Finally, the server sends the refined model back to all the cluster heads for further training rounds. We call this method *weight sharing method*. We performed extensive experiments on *weight sharing method*. However, we couldn't find much improvement. Even after clustering, the user data were not less sparse.

To reduce the sparsity issue in *weight sharing method*, we adopt the data sharing concept from paper [18]. In this paper, a certain but small portion of server data is shared with all the user devices, and the initial server model is trained first with the public dataset. Inspired by this method, we proposed a new method named *data sharing method*, where the whole public server data will be shared with the client devices and no server-side training and refining will happen. First of all, the server data is divided into an equal number of partitions as the total number of cluster heads. Then each partition is shared with each cluster head and the data is merged with the cluster head data maintaining time-step consistency. Thus no public dataset remains on the server. Merging the server data with the cluster head data greatly reduces the high sparsity. Now, a random model is initialized and sent to all the cluster heads. Each cluster head trains the model with its data and sends the trained model to the server. The server aggregates the models using an aggregation algorithm and sends the aggregated model back to all the cluster heads for the next round of training. We call this method *data sharing method*. We find

significant improvement in training compared to *weight sharing method*. In traditional federated learning, in server-side aggregation client models are aggregated by taking a weighted sum where weight is the ratio of client sample size and total sample size. However, in our problem, the client's crime count ratio is much better weight than the sample ratio. Therefore, We have also introduced a novel aggregation algorithm where we replace the sample ratio with client crime ratio in server side weighted aggregation.

1.3 Contribution

The contributions of this thesis are summarized as follows:

- We have proposed a new framework for crime prediction where we have utilized the Federated Learning approach to train the crime prediction model. Thus, we have ensured the data privacy of users.
- We have adopted a novel clustering based data-sharing method in federated learning where the server dataset is shared with the user dataset to enhance user data samples and obtain better results than the traditional federated learning settings.
- We have introduced our novel *FedProx with Modified Weighted Aggregation* algorithm in federated learning.
- We have run extensive experiments on different federated learning settings to show comparative analysis between different federated learning frameworks and the effect of different parameters.

1.4 Organization

In the next chapters, we provide detailed information about our thesis work. In Chapter 2, we discuss the related works on recent crime prediction models and federated learning works. In Chapter 3, we discuss our model, two different methods: *weight sharing method* and *data sharing method*, the architecture of FedCrime and four aggregation algorithms. In Chapter 4, we perform analysis of our experiment results. We discuss the experiment setup first and then related methods, datasets, hyperparameters, and measures of accuracy (MAE/MSE). Then we discuss the results we have found in different settings e.g. varying dataset sizes, varying cluster sizes, etc. Finally, in Chapter 5, we conclude our thesis with future work on federated learning-based crime prediction.

Chapter 2

Related Works

In this chapter, we describe the related works on both crime prediction and federated learning. For crime prediction, previous works include two major approaches: Statistical Machine Learning Approach and Deep Learning Approach. For federated learning, we will describe various aggregation functions, data partitioning-based horizontal federated learning approaches and various clustering methods used in federated learning frameworks.

2.1 Crime Prediction Models

Due to the large amount of available crime occurrence data various approaches have been taken to infer future crime occurrence. These approaches have created different kinds of machine learning models, and deep learning models.

2.1.1 Statistical and Machine Learning Models

The Negative Binomial variant of the poisson-based regression model is utilized by [2] to model the crime inference problem. In another study [3], the author used two new features - POI and Taxi flow data along with traditional crime features in Linear regression method and Negative Binomial Regression methods.

The random forest method is used by [4] to infer crime hotspots emphasizing on spatial correlation. In [19], the author used random forest based ensemble machine learning model to utilize the correlation between crime and various urban indicators such as illiteracy rate to infer crime.

In [5], the author used Crime hotspot mapping for crime prediction using KDE. Marching Square Kernel Density Estimation [6] introduces fast and accurate crime hotspot mapping for crime prediction.

The Autoregressive Integrated Moving Average (ARIMA) method is used by [7] to use the correlation between the crime incident and time.

Space-time variant of kernel density estimation is used by [8] to visualize 3D crime incident cube to understand the crime hotspot mapping in both spatio-temporal dimension. To capture spatio-temporal correlation of crime incidents [9] use techniques like cross-correlation measures and eigenvalue spectrum analysis.

2.1.2 Deep Learning Models

The availability of data and the success of traditional machine-learning models encouraged researchers to use various deep learning-based spatial-temporal models to capture crime occurrence patterns in recent times.

ST-SHN [13] uses a hypergraph learning paradigm to capture dynamic spatiotemporal crime patterns and also incorporate a multi-channel routing mechanism to capture time based dynamic relation between each crime types.

ST-HSL [12] used cross-region hypergraph structure learning and dual stage self supervise learning paradigm to learn region wise crime dependency and spatial temporal crime patterns.

In [20], Graph convolutional Gated Recurrent Unit (Graph-ConvGRU) is used to capture spatial-temporal and categorical relation between crime occurrences.

STMEC [21] uses a meta path guided crime prediction framework that captures the dynamic patterns of crime behaviors and also incorporates mutual interaction between social and environmental factors.

HAGEN [10] uses an adaptive graph learning module along with DCGRU (Diffusion Convolution Gated Recurrent Unit) and Graph Neural Network to capture crime-based relation between region and also capture temporal correlations.

AIST [11] created an interpretive crime prediction model using various GAT (Graph Attention Network) and SAB-LSTM model to capture spatio-temporal crime occurrence pattern. It also includes extra feature (Taxi trip data) by using fGAT.

2.2 Federated Learning

Federated learning is a decentralized approach to training a machine learning model which was first introduced by McMahan et al [1]. It assumes a client-server architecture where data resides in client devices. Each client trains local models on local data and sends their trained model to the server. The server generates a global model by aggregating the local models. The training

usually happens in several rounds of communication where in each round client devices train a local model and the server aggregates them into a global model. The major advantage of federated learning is preserving data privacy since user data is never transferred to the server.

Extensive research has been conducted to address the most common challenges in federated learning such as tackling client data variability and heterogeneity, designing the appropriate model aggregation technique, and increasing global model performance without sacrificing user privacy. Below, we present a summary of the existing works that are most relevant to our research problem.

2.2.1 Data Partitioning

One important issue is how the data is distributed or partitioned among the clients. In horizontal partitioning, client data has all the features but have different data rows respective to the client. In contrast, in vertical partitioning, each client have a subset of features of the original data. In our crime prediction problem, the crime incidences are distributed among the clients where the number of crimes faced by a client constitute the single feature of the data. Thus, it mostly aligns with horizontal partitioning. There are some security problems which are frequently observed in horizontal federated learning. Therefore researcher has come up with various idea resolve the problem.

Kim et al. [22] developed BlockFL, in which the block chain network is utilized by each mobile devices to update the local learning model.

Smith et al. [23] introduced the MOCHA federated learning method to address security challenges in multitasking. This approach enables collaboration across multiple sites to accomplish tasks while maintaining privacy and security.

In [24], an adaptive horizontal federated learning is developed which is used for Demand Response Baseline Load Estimation.

2.2.2 Aggregation Functions

The method of model aggregation is another important research question. The initial approach is to train the local model in conventional ways, compute the mean or average of the local model weights, and then use it as the global model. This conventional aggregation method performs really well in standard problem settings. However, in complex scenarios such as non-IID settings, conventional training in the client device and simple averaging fails to achieve the desired performance by the global model. Thus, techniques are devised to adjust the client training in several ways such as using special regularization technique or adapting learning rate with an aim to improving the global model.

FedAvg [1] function in each round takes the weights of the clients and updates the centralized weights by taking the weighted average of clients' weights.

$$f := \frac{1}{N} \sum_i n_i \cdot \omega_i$$

Here, $N = \sum_i n_i$ and n_i is the training sample size of i th client. Due to its simplicity, FedAvg's computation overhead is low and the central weights converge faster when clients' data distribution is IID. But FedAvg requires more communication overhead when clients' data distribution is Non-IID.

FedAvgM [25] uses momentum on FedAvg's aggregation function in server side. It gives better results than FedAvg in the case of non-iid clients data.

SCAFFOLD [26] showed that a slow convergence rate due to client data heterogeneity can be improved than FedAvg [1] by correcting the local updates by introducing control variates which will reduce client drift independent of client selection.

FedOPT [27] introduces optimizers in both the server and client. In each round, SGD is used in client-side optimization. Then each client sends their gradient and weights to the server. Server uses optimizers to update the weights for the next round. In server Adagrad, Adam, Yogi were used for three different algorithms respectively FedAdagrad, FedAdam, FedYogi.

FedProx [28] emphasized data heterogeneity and system heterogeneity problems in federated learning. Each client uses a proximal term in the local objective function to restrict the local updates to closer to the global model as much as possible. The proximal term is computed based on the l_2 -norm distance between the current global model and the local model and thus the local update is being limited by the proximal term. That is how it is trying to minimize the data heterogeneity problem. Conversely, it is natural that the system resources of all clients won't be similar. So, each client can't perform the same amount of local computation. Instead of discarding the device, FedProx [28] allows each device to find an inexact solution λ_k^t for client device k in round t .

FAFED [29] further improves the idea of FedOPT [27]. The author first proves that the simple adaptive method might converge but it doesn't resolve the heterogeneous data distribution problem. FAFED uses a shared adaptive learning rate so that each client's model doesn't diverge from the aggregated model. FedDC [30] also focused on the significant statistical heterogeneity among the data distributions on client side and to reduce the inconsistency between locally optimized client models, FedDC introduced an auxiliary local drift variable in each client to track the gap between the local model parameters and global model parameters. Basically, it utilizes the local drift variable to learn the gap between local and global model and updates the local objective function of each client, corrects the local model parameters before the central aggregation

process. FedNP [31] explored the fact that client models didn't explicitly consider global data distribution in previous federated learning models. These models change the local objective function so that the local model converges to a global model. FedNP passes a latent variable to clients that determine global data distribution approximation at the start of each round. Each selected client updates the latent variable in local training steps and sends the latent variable to the server. The server sends the aggregated latent variable along with weights for the next round.

Similar to FedDC [30] and FedProx [28], FedLA [32] introduced online Laplace approximation on both client and server models, a multivariate Gaussian product mechanism on server model to reduce the central aggregation errors for heterogeneous data settings of clients and also a loss passing mechanism from server to client to control local training on clients. Newly, FedLA came up with a solution, focusing on the posteriors on both server and client models, classifying the posterior problem as aggregation error and local forgetting.

FedPA [33] focused on inferring the global posterior distribution, where each client has the posterior of their data. So instead of the usual loss minimization technique, FedPA formulated federated learning as a posterior inference problem where local posterior inference is happening on the client side and global posterior inference is taking place on server side computation. FedPA also worked on computation and communication efficiency.

2.2.3 Clustering

Different approaches have been taken to resolve non-IID problem in federated learning. Clustering is one of the approaches where clients of the federated learning system form a cluster and share data and one of the clients from the cluster is chosen for training the model. Sattler et. al [17] first proposed the idea of clustered federated learning.

In [34], the clustering method has been used to handle data heterogeneity problems to develop a personalized stress monitoring system.

ClusterFedMet [35] used clustering to reduce communication overload of model training and developed a federated recommendation model.

FedSoft [36] trains both locally personalized model and high quality clustered model.

2.3 Summary

Crime prediction research employs both Statistical Machine Learning and Deep Learning approaches. Statistical methods like Negative Binomial regression [2] and Random Forest models [4] focus on spatial correlation and hotspot identification. Techniques such as Kernel Density

Estimation [5] and Marching Square mapping [6] provide detailed crime hotspot visualizations. Spatiotemporal correlations are captured through cross-correlation measures and eigenvalue spectrum analysis [8] [9].

Deep Learning models like ST-SHN [13], ST-HSL [12], and Graph Convolutional Gated Recurrent Unit (Graph-ConvGRU) are used for complex crime pattern analysis [20]. STMEC [21] integrates social and environmental factors, while HAGEN [10] combines adaptive graph learning with dynamic graph convolutional units and Graph Neural Networks for crime relationship modeling. AIST [11] utilizes Graph Attention Network and SAB-LSTM models with additional data like taxi trips to enhance prediction accuracy. These approaches aim to improve proactive crime prevention strategies by predicting future crime occurrences more accurately.

Federated learning, a decentralized approach where client devices train local models on their data without sharing it centrally. The server aggregates these local models to create a global model, ensuring data privacy as sensitive information remains on clients. Challenges include handling data variability and heterogeneity among clients, optimizing model aggregation techniques, and improving global model performance without compromising privacy. The research addresses these challenges with methods like BlockFL [22] using blockchain for secure updates, MOCHA [23] for multitasking security, and adaptive techniques [24] tailored to specific applications like Demand Response Baseline Load Estimation. These advancements aim to enhance the security and efficiency of federated learning across distributed networks.

In the federated learning settings, FedAvg [1] simply calculates the weighted average of all the training parameters from the clients. FedAvgM [25], SCAFFOLD [26], FedProx [26], FAFED [29], FedDC [30], FedNP [31] worked on the problem of non-IID data distribution. FedAVgM [25] introduced a momentum variable in the server-side aggregation. SCAFFOLD [26] worked on improving the client-side parameter updates while FedOPT [27] introduced different optimizers (FedAdagrad, FedAdam, FedYogi) [27] for server-side parameter optimization. FedProx [28] also focused on the problem of system heterogeneity so that the resources available on different devices could be used effectively. FAFED [29], FedDC [30] and FedNP [31] worked on improving the client-side model divergence by introducing an adaptive learning rate, local drift variable, and a latent global distribution approximation variable respectively. FedLA [32] used a multivariate Gaussian product mechanism on server-side model to reduce the central aggregation errors for heterogeneous data settings and FedPA [33] worked on global posterior distribution inference.

Various approaches in federated learning address the challenge of non-IID data distribution through clustering methods. Clustered Federated Learning (CFL), introduced by Sattler et al. [17], forms clusters among clients where each cluster shares similar data characteristics and one client per cluster trains the model, enhancing data homogeneity. In another application, clustering methods are utilized to develop a personalized stress monitoring system, en-

sureing tailored model training based on individual data characteristics. ClusterFedMet [35] reduces communication overload by clustering clients during model training and optimizing federated recommendation systems. FedSoft [36] combines locally personalized models with high-quality clustered models to improve overall performance and adaptability in federated settings. These approaches demonstrate how clustering enhances data efficiency, model quality, and personalization in federated learning while addressing challenges posed by non-IID data distributions.

Chapter 3

Our Model

In this chapter, we present our approach to solve the crime prediction problem in a federated learning setting. In Section 3.1, we discuss the system overview of FedCrime. In Section 3.2, we discussed how we integrated *clustering method* in traditional federated learning framework. Then in Section 3.3, we present the client-side components and the server-side components in FedCrime. In Section 3.4, we present our first approach called *weight sharing method* that closely aligns with the traditional federated learning approach with model aggregation. In Section 3.5, we present an improved approach called *data sharing method* that uses data sharing by the server to the clients. In Section 3.5.4 under Section 3.5, we showed the architecture of FedCrime and described different portions of the architecture briefly. Finally in Section 3.5.5 under Section 3.5, we presented two famous aggregation algorithms, FedAvg [1] and FedProx [28] and also our novel aggregation algorithms *FedProx with Modified Weighted Aggregation* and *FedAvg with Modified Weighted Aggregation*.

3.1 System Overview

Our system consists of a server and a large number of geographically distributed clients. The clients are the sources of data as they face crime incidents in their daily life. We assume that some clients share their data with the server and some clients do not. It is very common in real world where a reasonable percentage of people are always happy to share their incidents with the public in an effort for global contribution. Thus, in our system, server has access to a portion of the data that is shared by the clients.

In our system, all clients do not participate in federated learning. This is because most of the clients are general people who are primarily the crime data sources but do not have required computational resource to participate in the learning process. Instead, we assume that each region has a designated client who acts as the federated learning coordinator for the region and

participate in the learning process. It is usually the client with the most available computational resource in a region.

Our system thus consists of a number of clusters. Each cluster comprises a cluster head (or coordinator) and a group of members who serve as the data source clients for the cluster. Since the cluster head participates in the federated learning process, it needs the local data from all the clients in the cluster. It is a realistic assumption that all members in a cluster trust the coordinator and share their private crime incidence data exclusively with the cluster head. However, neither the clients nor the cluster head share the same data with the global server.

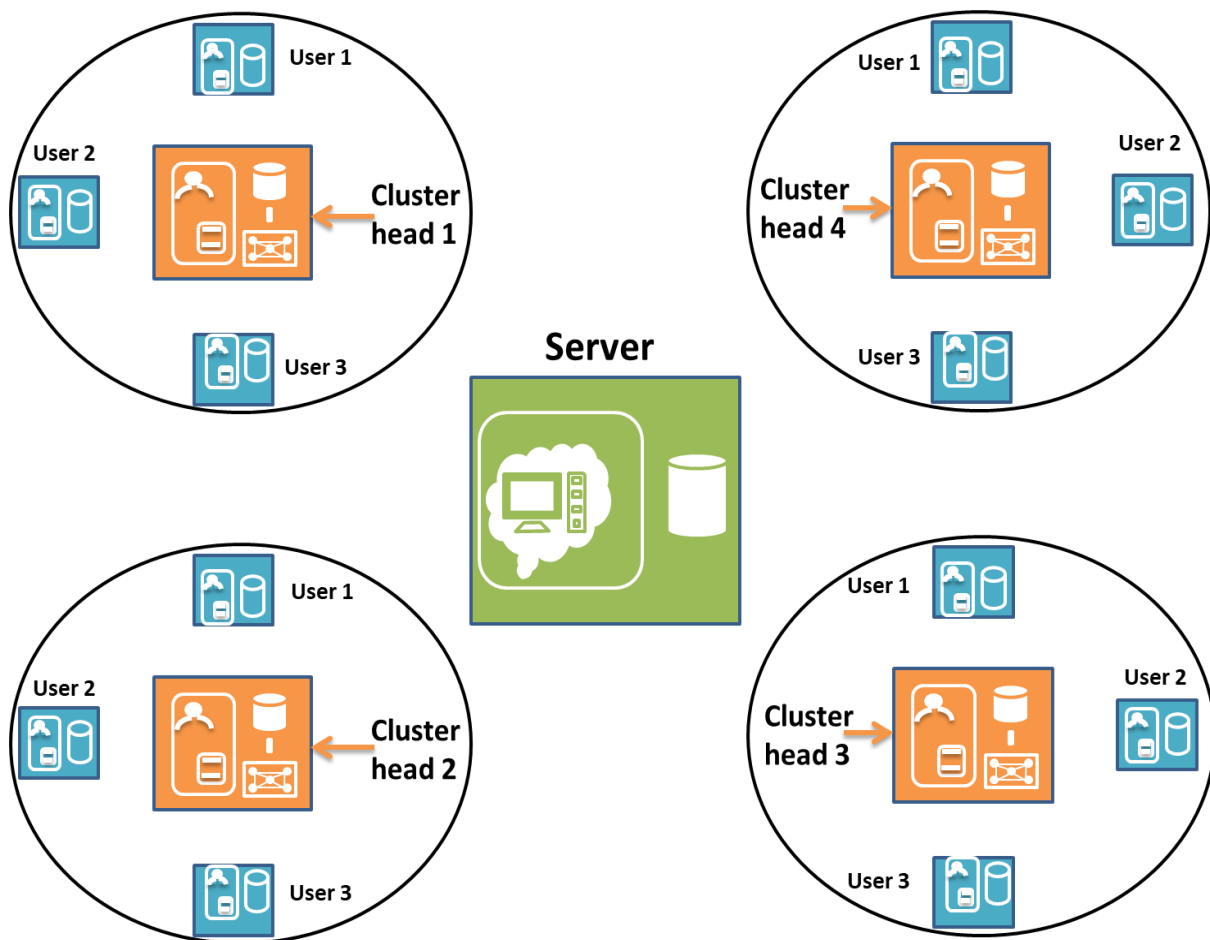


Figure 3.1: System Overview of FedCrime

Figure 3.1 shows the system overview of our approach. Here we can notice there is a server in the center of the system. The server has public crime dataset and surrounding the server, there are clusters. In each cluster, there are certain number of client devices and a cluster head in the middle of clusters. The circular cluster shape denotes that a cluster is formed within a trusted circle where user devices can securely share their data with the cluster head.

In summary, the entire crime data in our system is distributed among the server and the cluster heads. The global server holds the publicly shared crime data by the clients. The private portion

of the client data that are not shared with the server are distributed among the cluster heads. Each cluster head possesses the private data of the clients in its region.

3.2 Clustering Method

As we have already discussed, the traditional federated learning framework does not directly align with our crime prediction scenario. In traditional federated learning, it is assumed that each client devices must have enough data to train the model with its own private data. But in our crime prediction scenario, most of the users have very less data and there are zero crime incidents in maximum time-step for most users. Thus, training with this highly sparse data will tend to poor crime prediction model. Therefore, we introduced *clustering method* [17] in federated learning. In clustering method, all the remote client devices are grouped into certain number of clusters. Each cluster will have a certain number of devices who trust each other. We can think of a cluster as a trusted circle. In each cluster, not all the client devices will participate in the training. Only a single cluster head is selected and all other devices share their data with the cluster head securely. Then the cluster head merges the data of all the devices with its own data maintaining time-step consistency. That's how the clustering process is happening.

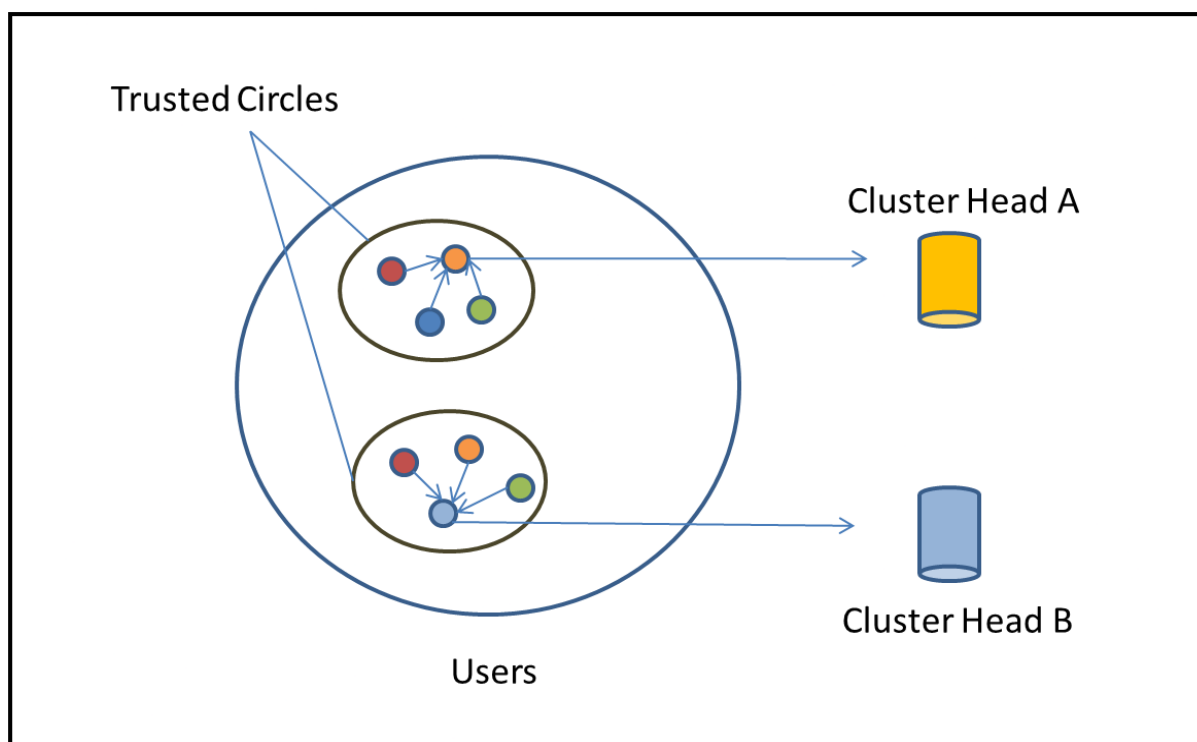


Figure 3.2: Clustering Method in Federated Learning

In Figure 3.2, we can see a circle denoting the entire set of users and two clusters forming with all the users for demonstration. In both clusters, all the users are sharing their data with the

cluster heads A and B. Therefore, only cluster head A and cluster head B will participate in further training process.

3.3 Client and Server Side Components

On the client side, in each cluster, we have a certain number of N clients each with their own data. And there are n clusters in our system. In each cluster, there is a cluster head in the center of the cluster. All the devices other than the cluster head will share their data with the cluster head. Therefore, n clients will have all the client data in total mathematically. Hence, we divided all the data of the N clients between these n cluster heads. There is a crime prediction model in the cluster head. The crime prediction model is working as a *black box* where any recent crime prediction model can be used.

On the server side, there is a crime prediction model similar to cluster heads. Here also crime prediction model is working as a *black box* so that we can fit any crime prediction model here. But the server-side crime prediction model and cluster heads' crime prediction model must be same. There is a public crime dataset in the server which the server will use for training the global model.

3.4 Weight Sharing Method

In this section, we will discuss about our first method *weight sharing method* where we tried to reduce the data sparsity in user devices by utilizing the *clustering method* in addition to the traditional federated learning method. The weight-sharing method [37–39] is an advanced approach in federated learning that combines traditional weight-sharing with a clustering mechanism to enhance the effectiveness of crime prediction models. This method introduces a multi-step process that involves training, aggregation, and refinement of crime prediction model to utilize both server and client datasets effectively.

In each training round, the process begins on the server, where the crime prediction model is initially trained using the server dataset. This dataset typically consists of publicly available crime data that serves as a general foundation for the model. The training on the server ensures that the model captures broad crime patterns and trends from diverse sources, which can be beneficial for initializing the model with a robust understanding of crime dynamics.

Once the server-side training is complete, the trained model is distributed to the cluster heads. In the context of clustering, each cluster head represents a group of clients that have been organized based on specific criteria, such as geographic location or similarity in crime patterns or connected in a trusted circle where each client trusts each other. The cluster heads then

train the model on their respective datasets. These datasets are aggregated versions of the data from all clients within the cluster, providing a more comprehensive and localized view of crime patterns specific to each cluster.

After completing the training on the client side, the cluster heads send their updated models back to the server. At the server, these models, now enriched with localized crime data, are aggregated using a specified algorithm. This aggregation process typically involves combining the model weights from each cluster head in a manner that ensures the resulting model reflects the contributions from all clusters. The aggregated model thus integrates diverse and localized crime data, making it more robust and comprehensive.

The server model is then re-initialized with this aggregated model, effectively updating it with the latest results gathered from the client-side training. This re-initialized model is then prepared for the next round of training, where the process repeats. The entire training cycle continues for a predefined number of rounds, allowing the model to iteratively improve and refine its crime prediction capabilities by continuously incorporating fresh data from both the server and the clients.

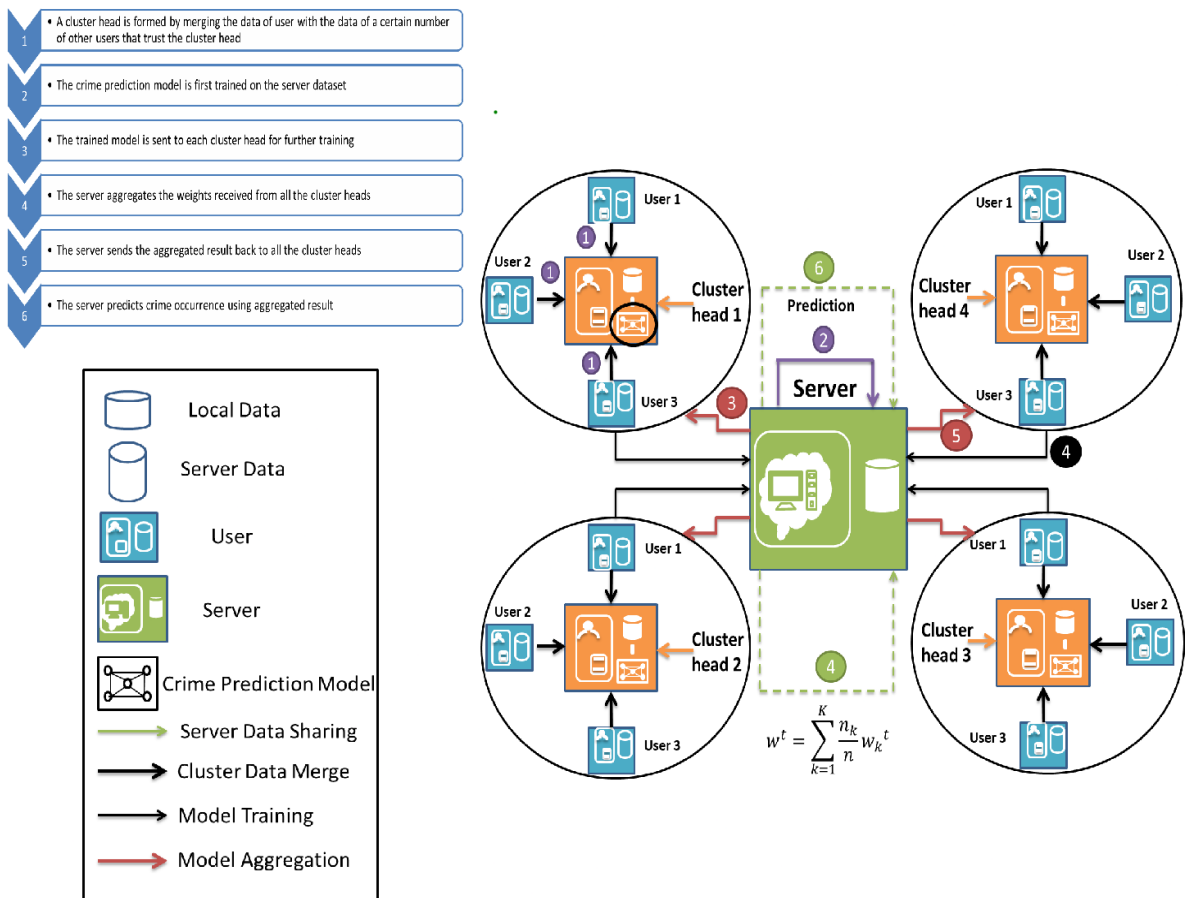


Figure 3.3: Weight Sharing Method

However, despite the advantages of the weight-sharing method, there is a significant limitation. Although clustering helps to organize and aggregate client-side data, a substantial portion of the data remains on the server side. The server data is typically more generalized and does not capture the specific, localized crime patterns that client data can provide.

A critical issue arises from the nature of the client datasets. Many client datasets exhibit sparsity in crime incidents, with numerous timestamps showing zero crime data. This sparsity means that for many time periods, there are no crime incidents reported by the clients, leading to insufficient data for training. Consequently, the models trained on these client datasets are not as effective, as they lack the comprehensive data required to capture the full spectrum of crime patterns.

Even with clustering, which aims to aggregate and enhance the client-side data, the fundamental problem of data sparsity remains. The aggregated data from the cluster heads may still be insufficient, as it predominantly consists of sparse crime incidents, leading to models that do not perform well. The localized models trained by the cluster heads struggle to generalize effectively due to the limited and incomplete data they are trained on.

As a result, the client-side trained models are not as performant as desired. The sparsity of crime data in client datasets leads to models that fail to capture the relation and trends necessary for accurate crime prediction. This limitation highlights a significant challenge in the weight-sharing method with clustering, where the imbalance and sparsity of data between the server and clients can hinder the overall performance and effectiveness of the federated learning system.

3.5 Data Sharing Method

In the traditional *weight sharing method* along with *clustering method* [17] doesn't create better samples in the client-side for client-side training. Most users have zero crime incidents data in most time-steps and even after *clustering method*, the aggregated samples have also zero data in maximum time-steps. Thus, *clustering method* alone can not solve the high data sparsity problem in our crime prediction scenario. Our major advantage is we have a notable portion of public crime data in our server. If we can merge the server data with our cluster heads' data maintaining time-step, then in each time-step, the zero data will reduce and the data sparsity problem will be solved further. Focusing on the major data sparsity issue, we introduce our novel *data sharing method*, where along with *clustering method*, we will use the server public data and share the data with cluster heads in clusters to reduce the zero data problem further.

3.5.1 Server Side Data Partitioning

In our novel *data sharing method*, we utilized server data sharing process with cluster heads. Assuming there are c clients who are the cluster heads. Each cluster head will have its data and its trusted clients' data. Thus, we divided all the client-side data into certain cluster heads, and these cluster heads participated in the client-side training.

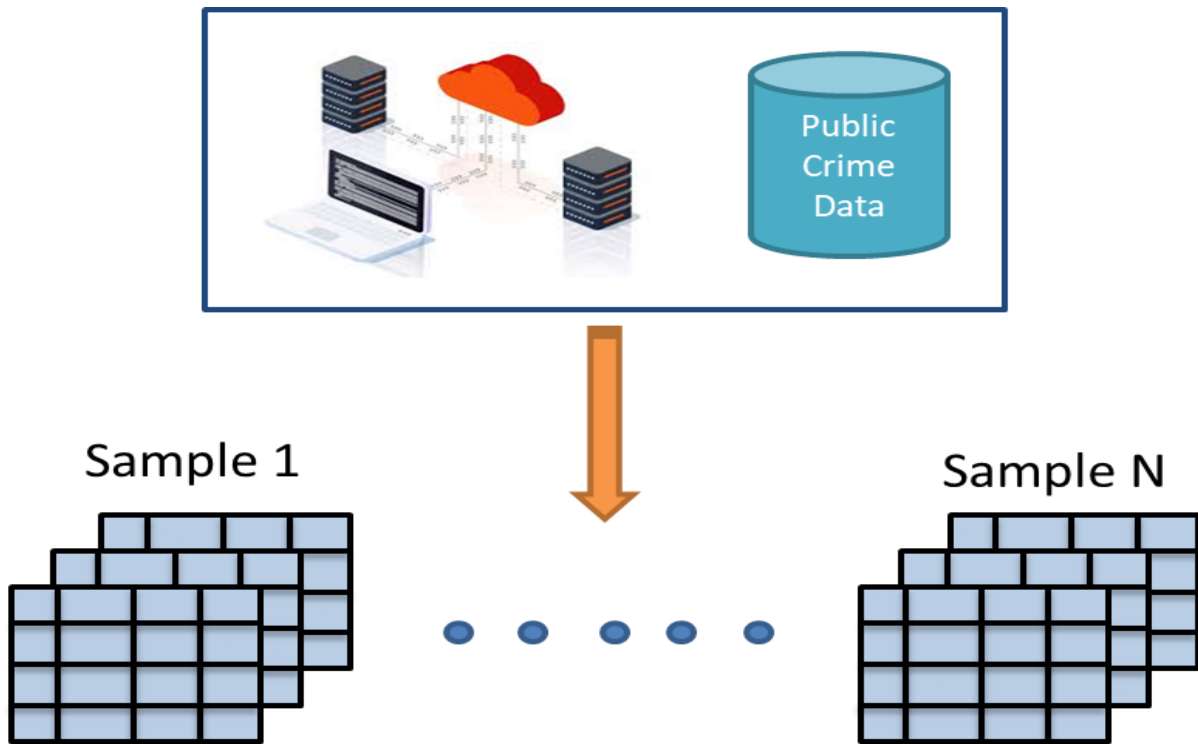


Figure 3.4: Server Side Data Partitioning

Then we shared the server data with the client data. For this purpose, we divided the server data samples into the equal number of partitions as the number of cluster heads.

3.5.2 Server Data Sharing with Cluster Heads

Each partition created from the server in the previous Section 3.5.1, was randomly shared with a cluster head. Then the server partition was merged with the cluster heads' data, maintaining the time-step consistency. Therefore, we have the client data and server data merged in each cluster head, maintaining each sample's temporal consistency.

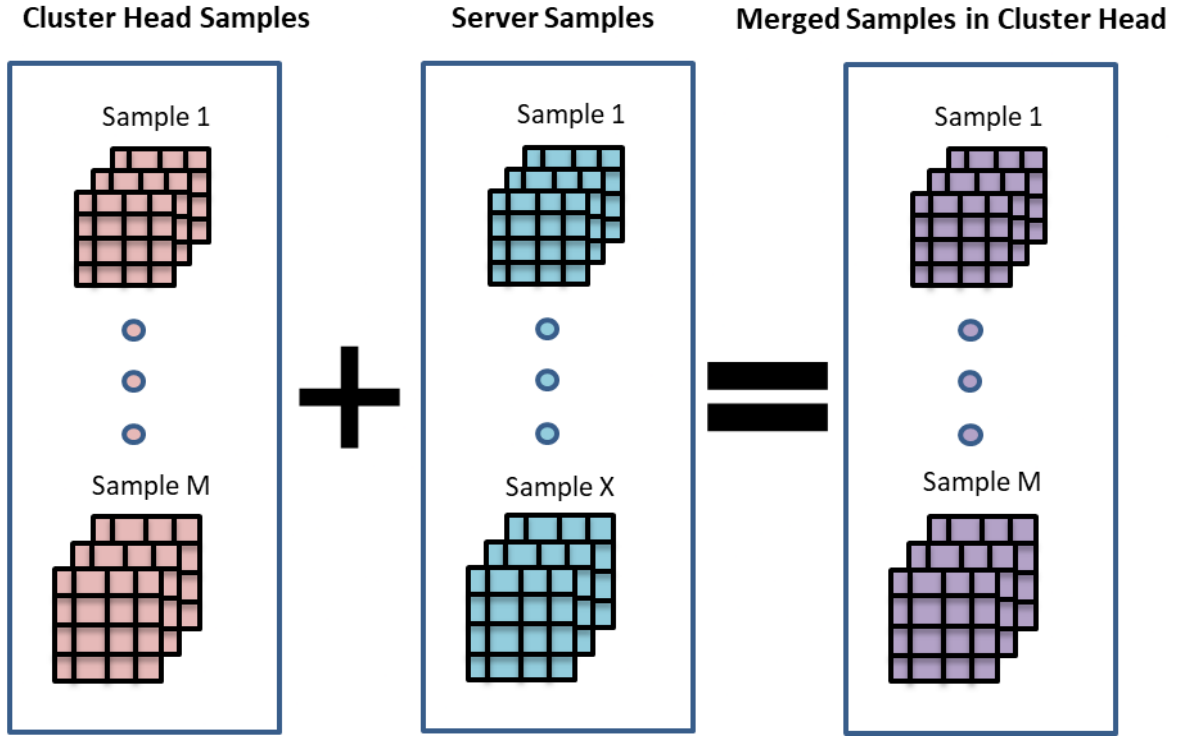


Figure 3.5: Server Data Sharing With Cluster Heads

3.5.3 Model Training and Weights Aggregation

After the data processing in the client-side cluster heads and server data sample partitioning followed by server data sharing with cluster heads, the cluster heads' model was trained for a certain number of rounds. In each round, each cluster head was trained on its own merged samples and the trained weights were sent to the central server. On the server side, the server received all weights and the server aggregated the weights using the weighted aggregation method of FedAvg [1] and FedProx [28].

In paper [18], researchers proposed a method similar to our *data sharing method*. In the above paper, a random percentage of the same global data is also shared with all the clients. Here they are not sharing the entire global data. The same portion of global data is shared with all the clients. They just added the server data with client data. There was no merging of data, maintaining temporal consistency in the paper. Researchers just added a common portion of data so that the trained model does not diverge much from the global model. This is one of the core differences between our novel *data sharing* approach and the related paper [18]. After the data sharing, the server model is initially trained with the global data, and the trained model is passed to clients for further rounds of training. But in our novel *data sharing method*, there is no central training in the server as there is no server data remaining in the server. All the server data is divided and shared with the cluster heads. The server aggregates the model parameters after all the cluster heads send the trained model to the server and this is the only functionality

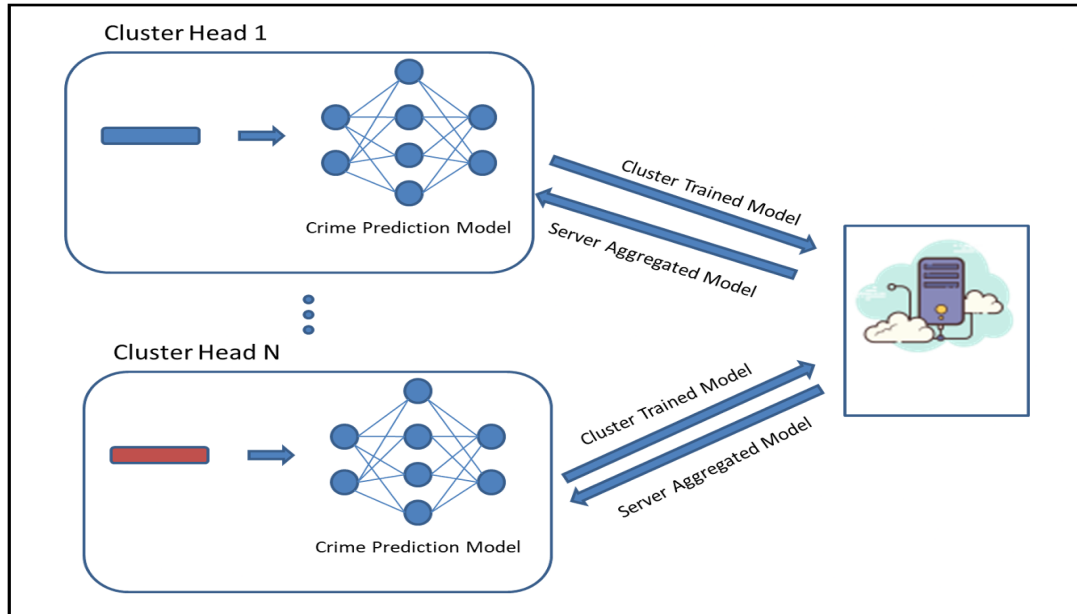


Figure 3.6: Model Training and Weights Aggregation

of server in our method. Therefore, in this approach also, our method *data sharing method* shows novelty from the method, mentioned in above paper.

3.5.4 Model Architecture

In the above figure, we showed the model architecture of our novel federated learning framework *FedCrime*. Here we can see, a central server with public dataset and all the clusters surrounding the server. For the crime prediction model in each cluster head, we adopted *AIST* [11] as this is a very recent and most accurate crime prediction model in our scenario. We have shown the crime prediction model in the upper right portion of the architecture. We can see a hGAT which is used to capture the spatial crime pattern of the crime incidents across certain regions. The embeddings coming from hGAT is passed to a SAB-LSTM to capture the temporal dynamics of the crime pattern and the final embedding from the SAB-LSTM is used for crime prediction. On top of the architecture, we have shown the steps of our novel *data sharing method* and this captures the entire *FedCrime* architecture.

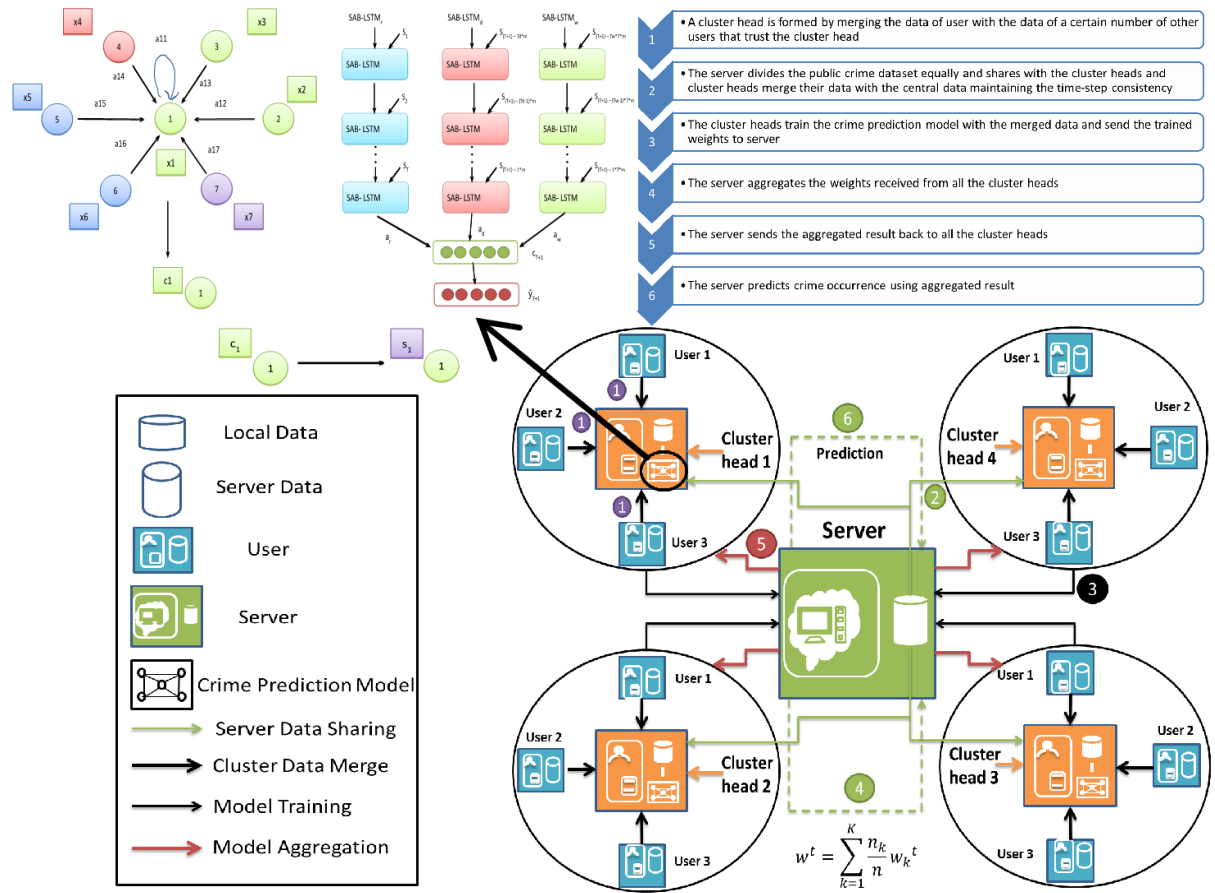


Figure 3.7: Model Architecture of FedCrime

3.5.5 Aggregation Algorithms

Algorithm 1: FedCrime with FedAvg

Input : K : total number of clients.

n_k : sample size for the k th client.

ρ : set of cluster heads.

x_i : set of clients in the trusted circle of the i th cluster head

T : total number of training rounds.

$ServerData_i$: server data portion that will be merged with cluster head i .

Output: θ^T : Final trained model.

```

1  $P \leftarrow \text{Length}(\rho)$ ;
2 for  $i \leftarrow 1$  to  $P$  do
3   |  $\text{MergeClusterHeadWithClientData}(i, x_i)$ ;
4 end
5 for  $j \leftarrow 1$  to  $P$  do
6   |  $\text{MergeClusterHeadWithServerData}(i, ServerData_i)$ ;
7 end
8  $\text{InitializeModelParameters}(\rho, \theta^0)$ ;
9 for  $t \leftarrow 0$  to  $T - 1$  do
10  | for  $p \leftarrow 1$  to  $P$  ( parallelly ) do
11    |  $\theta_p^{(t+1)} \leftarrow \text{argmin}_w F_p(w; w^t)$ ;
12  | end
13  |  $\theta_{avg}^{(t+1)} = \frac{\sum_{p=1}^P n_p \cdot \theta_p^{(t)}}{\sum_{p=1}^P n_p}$ ;
14  |  $\text{BroadcastAggregatedModel}(\theta_{avg}^{(t+1)})$ ;
15 end

```

Algorithm 1 shows the pseudocode for **FedAvg Algorithm** [1]. The input of the algorithm are total number of clients K , the sample size of each of the k th client n_k , set of cluster heads ρ , set of clients in the trusted circle of the i th cluster head x_i , total number of training rounds T . The algorithm's output is the final trained model after T th round, θ^T .

In line 1, $\text{Length}(\rho)$ function returns the length P of the set of cluster heads ρ . Then in $\text{MergeClusterHeadWithClientData}(i, x_i)$ in line 3, each i th cluster head merges its data with the crime data of all the clients in the set of clients x_i in the trusted circle of that cluster head. Further server crime data is divided into P equal partitions. Each server data partition $ServerData_i$ is merged with the data of each cluster head i maintaining consistency in timesteps, in line 6 in $\text{MergeClusterHeadWithServerData}(i, ServerData_i)$. Finally, the data-sharing process ends and the crime prediction model is randomly initialized with random parameters and the initial model is broadcasted to all the cluster heads in line 8.

Then the crime prediction model trains for T number of rounds. In each round, all the models in each cluster head parallelly train the model and update the model parameter as $\theta_{avg}^{(t+1)}$ for the next training round. Finally, in line 13, all the trained model parameters are aggregated in the server using FedAvg [1] aggregation algorithm and the server broadcasts the model to all the cluster heads for next round of training in line 14.

Algorithm 2: FedCrime with FedAvg Weighted Aggregation

Input : K : total number of clients.

n_k : sample size for the k th client.

ρ : set of cluster heads.

x_i : set of clients in the trusted circle of the i th cluster head

T : total number of training rounds.

$ServerData_i$: server data portion that will be merged with cluster head i .

Output: θ^T : Final trained model.

```

1  $P \leftarrow \text{Length}(\rho);$ 
2 for  $i \leftarrow 1$  to  $P$  do
3   |  $\text{MergeClusterHeadWithClientData}(i, x_i);$ 
4 end
5 for  $j \leftarrow 1$  to  $P$  do
6   |  $\text{MergeClusterHeadWithServerData}(i, ServerData_i);$ 
7 end
8  $\text{InitializeModelParameters}(\rho, \theta^0);$ 
9 for  $t \leftarrow 0$  to  $T - 1$  do
10  | for  $p \leftarrow 1$  to  $P$  ( parallelly ) do
11    |  $\theta_p^{(t+1)} \leftarrow \text{argmin}_w F_p(w; w^t);$ 
12  | end
13  |  $\theta_{avg}^{(t+1)} = \frac{\sum_{p=1}^P \epsilon_p \cdot \theta_p^{(t)}}{\sum_{p=1}^P \epsilon_p};$ 
14  |  $\text{BroadcastAggregatedModel}(\theta_{avg}^{(t+1)});$ 
15 end
```

Algorithm 2 shows the pseudocode for **FedAvg with Weighted Aggregation Algorithm**. All the steps that are explained in this algorithm are similar to the algorithm 1, except the line 13. Here in line 13, the weight is the ratio of user crime count and total crime counts of all users. Therefore, we are prioritizing the user with more crime incidents as the model trained in a user with more crime incidents in each time-step will provide better results.

Algorithm 3 shows the pseudocode for **FedProx Algorithm** [28]. All the steps that are explained in this algorithm are similar to the algorithm 1, except the line 11. Here an additional code added in line 11. In **FedProx** [28], a regularizer term $\frac{\mu}{2} \|w - w^t\|^2$ is added with client-side

Algorithm 3: FedCrime with FedProx

Input : K : total number of clients. n_k : sample size for the k th client. ρ : set of cluster heads. x_i : set of clients in the trusted circle of the i th cluster head T : total number of training rounds. $ServerData_i$: server data portion that will be merged with cluster head i .**Output:** θ^T : Final trained model.

```

1  $P \leftarrow \text{Length}(\rho)$ ;
2 for  $i \leftarrow 1$  to  $P$  do
3   |  $\text{MergeClusterHeadWithClientData}(i, x_i)$ ;
4 end
5 for  $j \leftarrow 1$  to  $P$  do
6   |  $\text{MergeClusterHeadWithServerData}(i, ServerData_i)$ ;
7 end
8  $\text{InitializeModelParameters}(\rho, \theta^0)$ ;
9 for  $t \leftarrow 0$  to  $T - 1$  do
10  | for  $p \leftarrow 1$  to  $P$  ( parallelly ) do
11    |  $\text{argmin}_w F_p(w; w^t) \leftarrow F_p(w) + \frac{\mu}{2} \|w - w^t\|^2$ 
12    |  $\theta_p^{(t+1)} \leftarrow \text{argmin}_w F_p(w; w^t)$ ;
13  | end
14  |  $\theta_{avg}^{(t+1)} = \frac{\sum_{p=1}^P n_p \cdot \theta_p^{(t)}}{\sum_{p=1}^P n_p}$ ;
15  |  $\text{BroadcastAggregatedModel}(\theta_{avg}^{(t+1)})$ ;
16 end

```

objective function $F_p(w)$ so that in client side training, the model does not diverge much from the global model. This term is called *Proximal Term*. The more value of the proximal term, the less it will diverge from the globally trained model. This is how *FedProx* algorithm helps the client-side training to converge with global model.

Algorithm 4: FedCrime with FedProx Weighted Aggregation

Input : K : total number of clients.

n_k : sample size for the k th client.

ρ : set of cluster heads.

x_i : set of clients in the trusted circle of the i th cluster head

T : total number of training rounds.

$ServerData_i$: server data portion that will be merged with cluster head i .

Output: θ^T : Final trained model.

```

1  $P \leftarrow \text{Length}(\rho)$ ;
2 for  $i \leftarrow 1$  to  $P$  do
3   |  $\text{MergeClusterHeadWithClientData}(i, x_i)$ ;
4 end
5 for  $j \leftarrow 1$  to  $P$  do
6   |  $\text{MergeClusterHeadWithServerData}(i, ServerData_i)$ ;
7 end
8  $\text{InitializeModelParameters}(\rho, \theta^0)$ ;
9 for  $t \leftarrow 0$  to  $T - 1$  do
10  | for  $p \leftarrow 1$  to  $P$  ( parallelly ) do
11    |  $\text{argmin}_w F_p(w; w^t) \leftarrow F_p(w) + \frac{\mu}{2} \|w - w^t\|^2$ 
12    |  $\theta_p^{(t+1)} \leftarrow \text{argmin}_w F_p(w; w^t)$ ;
13  | end
14  |  $\theta_{avg}^{(t+1)} = \frac{\sum_{p=1}^P \epsilon_p \cdot \theta_p^{(t)}}{\sum_{p=1}^P \epsilon_p}$ ;
15  |  $\text{BroadcastAggregatedModel}(\theta_{avg}^{(t+1)})$ ;
16 end

```

Algorithm 4 shows the pseudocode for **FedProx with Weighted Aggregation Algorithm**. All the steps that are explained in this algorithm are similar to the algorithm 3, except in the line 14. Here in line 14, the weight is the ratio of user crime count and total crime counts of all users. Hence, we are prioritizing the user with more crime incidents as the model trained in a user with more crime incidents in each time-step will provide better results.

Chapter 4

Experiments

In this chapter, we discuss about our overall experiments and results. In Section 4.1, we discuss overall experiment setup. In Sections 4.1.1, 4.1.2, 4.1.3 and 4.1.4, we present a description of the dataset, methods, hyperparameters, and performance measurement metrics used in experiments. After experimental setup, in result and discussion Section 4.2, we showed the results of our experiments and discussed them.

4.1 Experiment Setup

In this section, we describe how datasets are being created in Section 4.1.1, which methods are being used for experimentation in Section 4.1.2. In Section 4.1.3, we discussed how hyperparameters are used and what values are used for those hyperparameters, and finally which measures are used for choosing the best model or comparing between models.

4.1.1 Datasets

We are using the public crime dataset of Chicago (2019) [40] and the day-to-day Foursquare check-in dataset used in [41]. For simulation purposes, we need private user crime data. Hence, we first split the public crime dataset into two portions, Server and Client. Then, We distribute the crime incident data to the user uniformly. For distributing, For each region we first select the set of users who have at least visited that region once which can be found from the Foursquare check-in dataset [41]. Then we take all crime incidents in that region from the client portion and assign a random user from the selected user. In this way, A user dataset will be created. However, from the crime dataset, it can be seen that some regions may not have enough crime incident data. As a result, some users will have insufficient data and, in some cases zero data. Therefore, We are choosing those regions for experiments that have enough data to distribute

among users. Furthermore, crime types other than Theft also have lower amount of incident records. Hence, we choose to do our experiment on Theft. Also, the same can be observed for the check-in dataset that some regions may not have enough user check-in. Hence, We are also choosing those regions that have more than a certain amount of users (e.g. 100 users) who have visited the region at least once. We chronologically split the dataset as 3 months for testing and 9 months for training.

4.1.2 Methods

We compare six different federated learning models - Two models of weight sharing method using FedAvg [1] and FedProx [28] and two data sharing method described in algorithm 1 and 3. And addition to these two data sharing methods we also experimented using FedAvg Data sharing Weighted Aggregation and FedProx Data Sharing Weighted Aggregation which are described in algorithm 2 and 4.

4.1.3 Hyperparameters

We use the same parameter settings as AIST [11] for our client-side model parameters except for the number of epochs which is 4 per round in our model. Also, we set the number of rounds to 20. We use $\mu = 0.01$ for the proximal term used in models based on Fed Prox. For default configuration, we set the cluster size to 50 and server-client data partition to 50%.

4.1.4 Performance Measures

We use mean absolute error (MAE) and mean squared error (MSE) for evaluating the prediction of the models.

$$\mathbf{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{Y}_i - Y_i| \quad \mathbf{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{Y}_i - Y_i)^2$$

Here, $\hat{Y}$ is the ground truth, and Y is the predicted value n is total sample size.

4.2 Results and Discussions

In this section, we show the results with table, chart and graph of the comparative assessment of different federated learning models, the effects of varying cluster head size, and the effects of varying server data size and discuss them in details.

4.2.1 Comparative Assessment

We have done experiments on 29 regions and for crime category theft using six federated learning models and compared the results. For this experiment, We have kept 50% of the data on the server and the rest resides among clients. Furthermore, We have set the size of the cluster head to 50. In table 4.1, the results are shown where weight-sharing methods and data-sharing methods are separated. We can see that the federated learning models that use the data-sharing method perform much better than models using weight sharing method. FedCrime achieves 7.46% lower MAE and 19.27% lower MSE compared to the traditional Weight Sharing method. Even when we changed the aggregation algorithm from FedAvg [1] to FedProx [28] the average result didn't change due to user data sparsity. However, when the data-sharing method is used, changes in the aggregation algorithm result in visible performance improvement.

Model		Theft	
		MAE	MSE
Weight Sharing	Fed Avg	0.576	1.66
	Fed Prox	0.576	1.66
Data Sharing	Fed Avg	0.535	1.34
	Fed Prox	0.534	1.315
	Fed Avg Weighted Aggregation	0.533	1.32
	Fed Prox Weighted Aggregation	0.533	1.33

Table 4.1: Comparison of Different Federated Learning Frameworks

4.2.2 Effect of Varying Cluster Size

We conducted experiments across 7 regions and varying cluster head sizes (10, 25, 50, 75, 100), with 50% of crime data remaining on the server and the rest distributed among users. Fig. 4.1 and Fig. 4.2 shows that as cluster size decreases, performance improves across four model using data sharing approach. The reason behind the improvement is that lower cluster size means there is higher amount of users in a single cluster which creates better samples (less sparsed) after merging with server samples. However due to not using server data in the client side the weight-sharing method performs the same across all cluster size. However, as cluster size decreases client privacy also decreases because client size in a cluster increases. We can also see that when cluster head count ranges between 25 to 75, data-sharing methods perform much better than weight-sharing methods within an acceptable privacy enhancement. As a result, we can obtain both better performance and improved privacy.

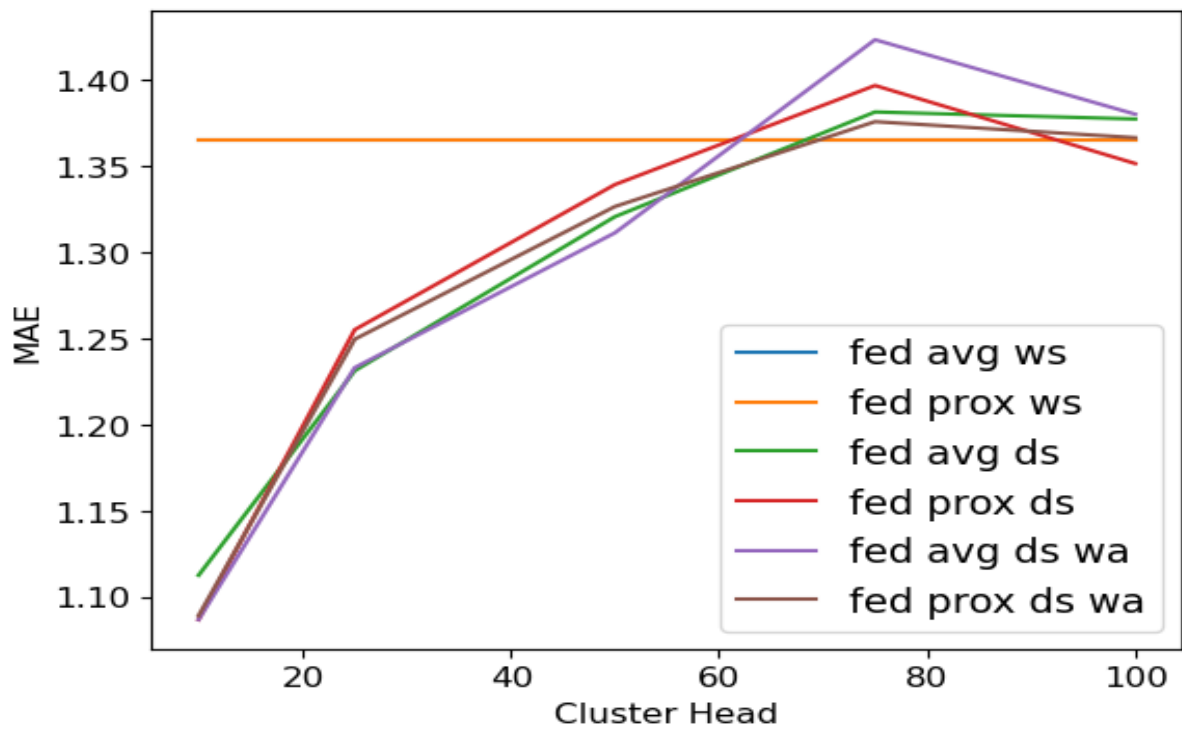


Figure 4.1: Effect of varying cluster size on MAE

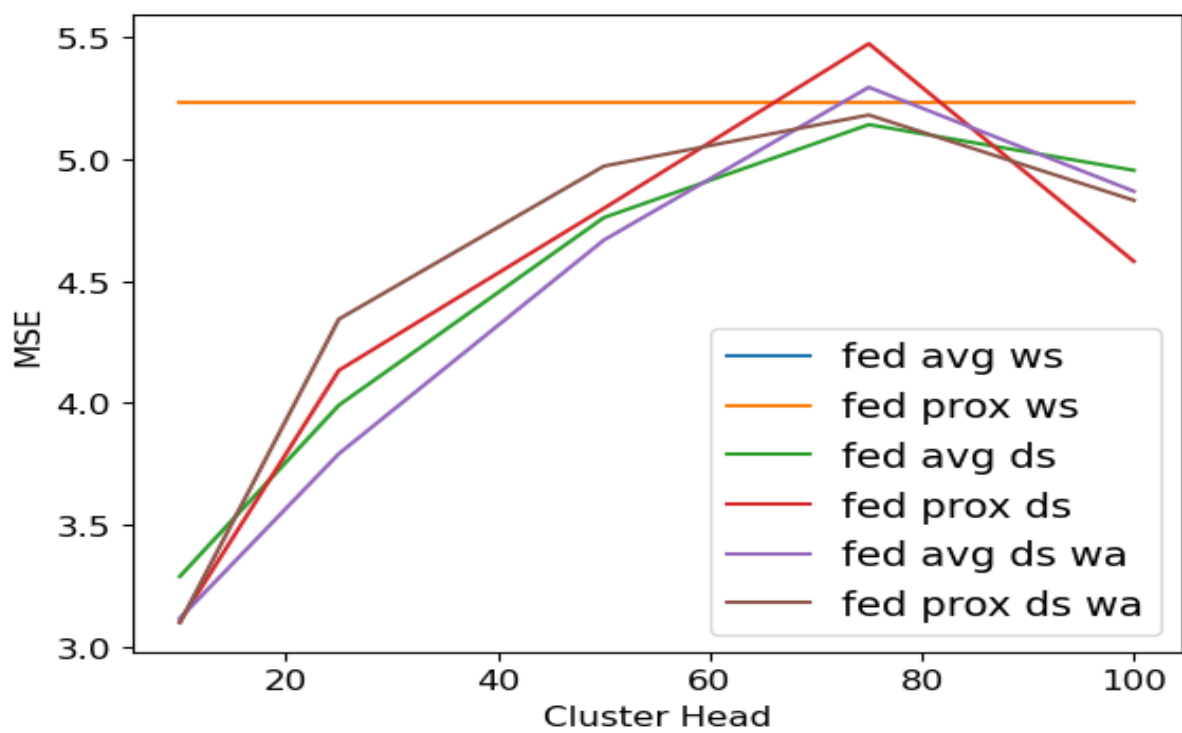


Figure 4.2: Effect of varying cluster size on MSE

4.2.3 Effect of Varying Server Dataset Size

We experimented across 7 regions, varying server data sizes (30%, 50%, 70%) and using 50 cluster head. Fig. 4.3 and Fig. 4.4 shows that our aggregation method works better than the

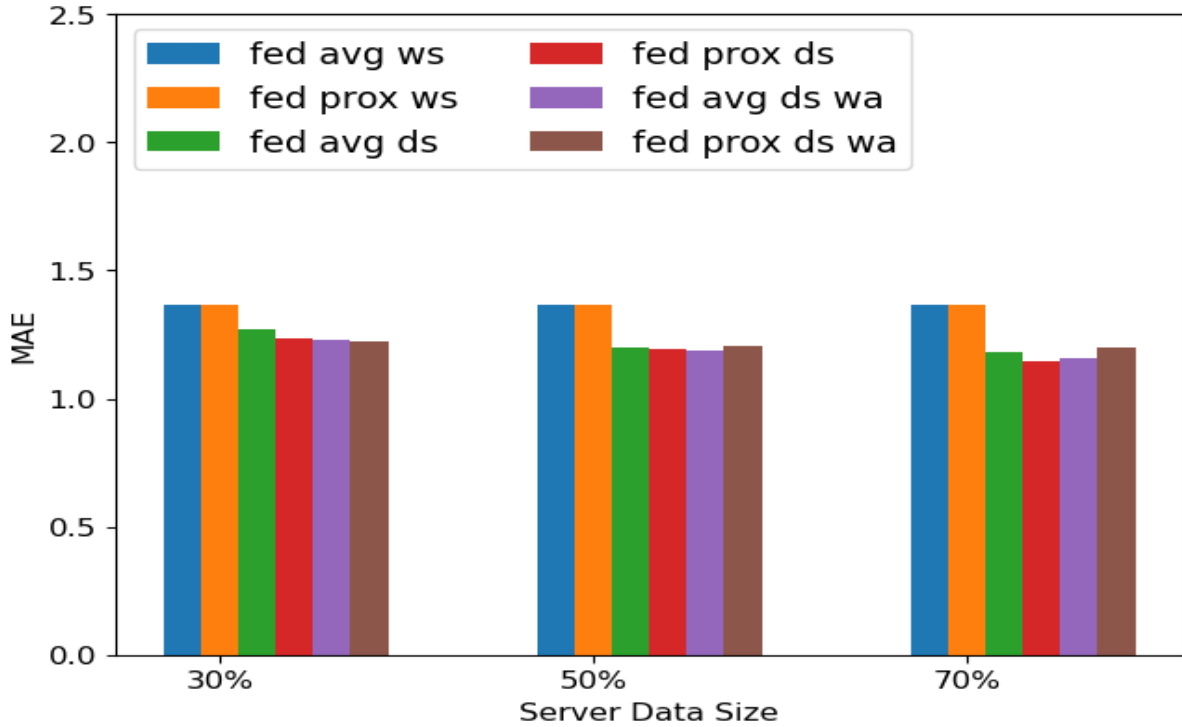


Figure 4.3: Effect of varying Server Datasize on MAE

traditional aggregation method in all server data sizes. Here, increased server data size improves model performance as expected because as server data increases, samples sent by server to cluster head also improves user data by decreasing sparsity. However, In our context it is highly likely that user as a whole will have a majority portion of crime data then server/centralized data. This scenario can be simulated when we have only 30% server data. In this scenario, FedCrime which uses our novel aggregating algorithm performs better than the model using traditional aggregating methods. This better performance is the result of giving priority to those user who have higher amount of crime data instead of user sample size as higher amount of crime counts means the user trained its model on much less sparse data. As a result, in server side aggregation, we are getting much better model also using FedProx [28] instead of FedAvg [1] resolves noniid problem of federated learning frameworks.

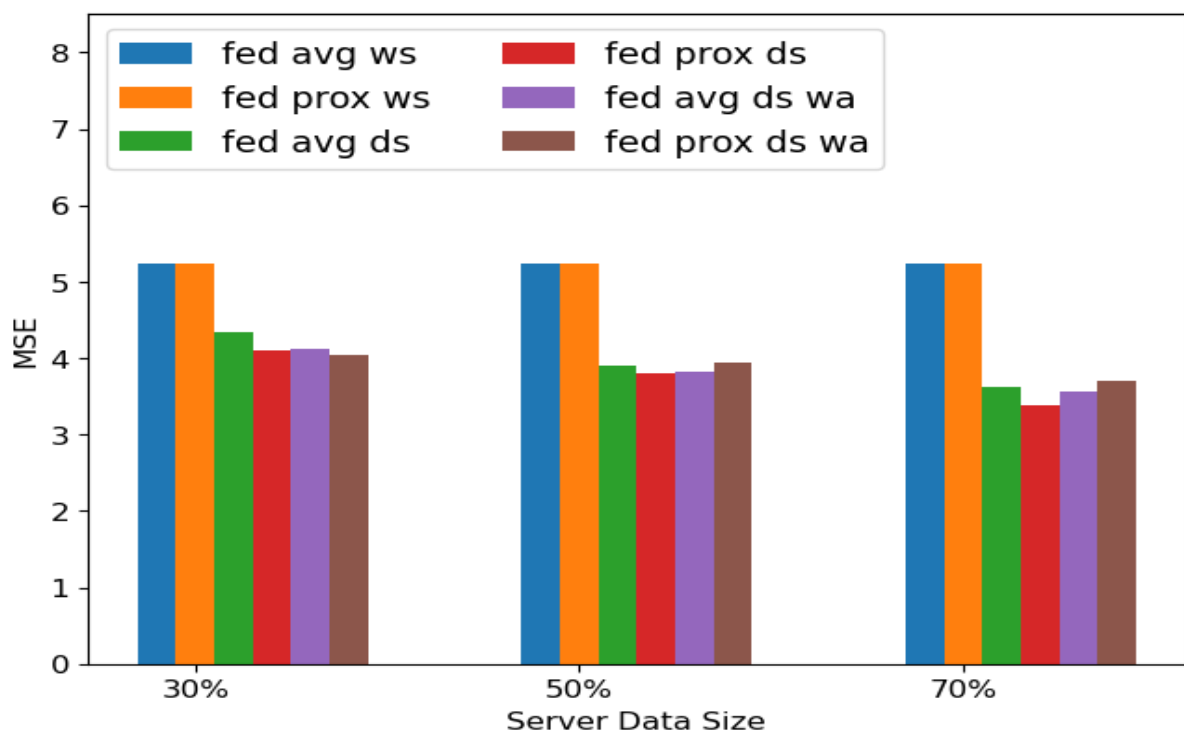


Figure 4.4: Effect of varying Server Datasize on MSE

Chapter 5

Conclusion and Future Work

Federated Learning can add a new dimension to crime prediction. Most of the works on crime prediction use only centralized crime dataset that is public. But the major problem is this public datasets are often incomplete and backdated. Due to privacy and security concern, most people are unwilling to share their crime incidents publicly. Furthermore, most people do not report crime on police stations. Hence, the public dataset do not contain most recent crime incidents. This is a serious concern and we need work to focus on capturing recent crime incidents but also keeping them private. Therefore, people do not need to worry about their privacy and at the same time our crime prediction will give more accurate predictions.

By using federated learning, we can take the advantage of keeping user data private and at the same time we can use crowdsourced data in model training. Therefore, we are getting the trained parameters from the models of the users, not the user data. That's how federated learning ensures privacy. But a problem remains that, traditional federated learning works are not directly applicable in case of crime prediction. As we are saying about ensuring privacy, we need to train models on user devices. But unfortunately, most user will have very less data in their devices for training. In the data samples in user devices, maximum samples will contain zero crime data. Hence, the final trained model will predict zero crime incidents will happen in future and this is a serious problem. By utilizing *clustering method* in federated learning, we can reduce the number of training devices and at the same time we can reduce the sample sparsity. Therefore, we tried our first attempt *weight sharing method*, where we clusters the devices and run the traditional federated learning approach. However, the data sparsity problem still became a severe issue, even after clustering. Finally, in our novel *data sharing method* where the server is sharing the public dataset with the users, we were able to reduce the data sparsity problem to some extent. Also we introduced our novel aggregation method *modified weighted aggregation in fedProx and fedAvg* for further improvement in FedCrime. We have run extensive experiments on various federated learning settings for both *data sharing method* and *weight sharing method* to show comparative analysis between different federated learning

frameworks. Our results show that our novel *data sharing method* outperforms the traditional *weight sharing method* with clustering. Furthermore, when cluster heads have much more data than the server, our novel aggregation method outperforms traditional aggregation methods.

More experiments need to be run to validate our result with different hyperparameter settings and for more crime categories to compare their results. Also experiments with different aggregation methods (e.g. FedNP [31], FedDC [30]) are needed to investigate whether further improvements can be made, with and without our novel *weighted aggregation* approach. Finally, different datasets can be used in **FedCrime** to compare with our current results for further investigations.

References

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, “Communication-Efficient Learning of Deep Networks from Decentralized Data,” in *Proceedings of International Conference on Artificial Intelligence and Statistics*, pp. 1273–1282, 2017.
- [2] D. Osgood, “Poisson-Based Regression Analysis of Aggregate Crime Rates,” *Journal of Quantitative Criminology*, vol. 16, no. 1, pp. 21–43, 2000.
- [3] H. Wang, D. Kifer, C. Graif, and Z. Li, “Crime Rate Inference with Big Data,” in *Proceedings of the ACM International Conference on Knowledge Discovery and Data Mining*, pp. 635–644, 2016.
- [4] S. Yao, M. Wei, L. Yan, C. Wang, X. Dong, F. Liu, and Y. Xiong, “Prediction of Crime Hotspots based on Spatial Factors of Random Forest,” in *Proceedings of International Conference on Computer Science Education*, pp. 811–815, 2020.
- [5] S. Chainey, L. Tompson, and S. Uhlig, “The Utility of Hotspot Mapping for Predicting Spatial Patterns of Crime,” *Security Journal*, vol. 21, no. 1, pp. 4–28, 2008.
- [6] J. F. de Queiroz Neto, E. M. d. Santos, and C. A. Vidal, “MSKDE - Using Marching Squares to Quickly Make High Quality Crime Hotspot Maps,” in *Proceedings of SIB-GRAPI Conference on Graphics, Patterns and Images*, pp. 305–312, 2016.
- [7] X. Shu, H. Yuan, and P. Chen, “Forecasting Crime Using the ARIMA Model,” in *Proceedings of International Conference on Fuzzy Systems and Knowledge Discovery*, pp. 627–630, 2008.
- [8] T. Nakaya and K. Yano, “Visualising Crime Clusters in a Space-time Cube: An Exploratory Data-analysis Approach Using Space-time Kernel Density Estimation and Scan Statistics,” *Transactions in GIS*, vol. 14, no. 3, pp. 223–239, 2010.
- [9] J. L. Toole, N. Eagle, and J. B. Plotkin, “Spatiotemporal Correlations in Criminal Offense Records,” *ACM Trans. Intell. Syst. Technol.*, vol. 2, no. 4, 2011.

- [10] C. Wang, Z. Lin, X. Yang, J. Sun, M. Yue, and C. Shahabi, “HAGEN: Homophily-Aware Graph Convolutional Recurrent Network for Crime Forecasting,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 4193–4200, 2022.
- [11] Y. Rayhan and T. Hashem, “AIST: An Interpretable Attention-Based Deep Learning Model for Crime Prediction,” *ACM Trans. Spatial Algorithms Syst.*, vol. 9, no. 2, 2023.
- [12] Z. Li, C. Huang, L. Xia, Y. Xu, and J. Pei, “Spatial-Temporal Hypergraph Self-Supervised Learning for Crime Prediction,” in *Proceedings of IEEE International Conference on Data Engineering*, pp. 2984–2996, 2022.
- [13] L. Xia, C. Huang, Y. Xu, P. Dai, L. Bo, X. Zhang, and T. Chen, “Spatial-Temporal Sequential Hypergraph Network for Crime Prediction with Dynamic Multiplex Relation Learning,” in *Proceedings of International Joint Conference on Artificial Intelligence*, pp. 1631–1637, 2021.
- [14] H. AlSalman, M. S. Al-Rakhami, T. Alfakih, and M. M. Hassan, “Federated Learning Approach for Breast Cancer Detection Based on DCNN,” *IEEE Access*, vol. 12, pp. 40114–40138, 2024.
- [15] S. Alawadi, K. Alkharabsheh, F. Alkhabbas, V. R. KEBande, F. M. Awaysheh, F. Palomba, and M. Awad, “FedCSD: A Federated Learning Based Approach for Code-Smell Detection,” *IEEE Access*, vol. 12, pp. 44888–44904, 2024.
- [16] M. Bhavsar, Y. B. Bekele, K. Roy, J. C. Kelly, and D. B. Limbrick, “FL-IDS: Federated Learning-Based Intrusion Detection System Using Edge Devices for Transportation IoT,” *IEEE Access*, vol. 12, pp. 52215–52226, 2024.
- [17] F. Sattler, K. R. Müller, and W. Samek, “Clustered Federated Learning: Model-Agnostic Distributed Multitask Optimization Under Privacy Constraints,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 8, pp. 3710–3722, 2021.
- [18] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, “Federated Learning with Non-IID Data,” *arXiv preprint arXiv:1806.00582*, 2018.
- [19] L. G. Alves, H. V. Ribeiro, and F. A. Rodrigues, “Crime prediction Through Urban Metrics and Statistical Learning,” *Physica A: Statistical Mechanics and its Applications*, vol. 505, pp. 435–443, 2018.
- [20] S. F. Tekin and S. S. Kozat, “Crime Prediction with Graph Neural Networks and Multivariate Normal Distributions,” *Signal, Image and Video Processing*, vol. 17, no. 4, pp. 1053–1059, 2023.
- [21] Y. Sun, T. Chen, and H. Yin, “Spatial-Temporal Meta-Path Guided Explainable Crime Prediction,” *World Wide Web*, vol. 26, no. 4, pp. 2237–2263, 2023.

- [22] H. Kim, J. Park, M. Bennis, and S.-L. Kim, “Blockchained On-Device Federated Learning,” *IEEE Communications Letters*, vol. 24, no. 6, pp. 1279–1283, 2020.
- [23] V. Smith, C. Chiang, M. Sanjabi, and A. Talwalkar, “Federated Multi-Task Learning,” in *Proceedings of Annual Conference on Neural Information Processing Systems*, pp. 4424–4434, 2017.
- [24] R. Wang, H. Qiu, H. Gao, C. Li, Z. Y. Dong, and J. Liu, “Adaptive Horizontal Federated Learning-Based Demand Response Baseline Load Estimation,” *IEEE Transactions on Smart Grid*, vol. 15, no. 2, pp. 1659–1669, 2024.
- [25] T.-M. H. Hsu, Qi, and M. Brown, “Measuring the Effects of Non-Identical Data Distribution for Federated Visual Classification,” *ArXiv*, vol. abs/1909.06335, 2019.
- [26] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, “SCAFFOLD: Stochastic Controlled Averaging for Federated Learning,” in *Proceedings of International Conference on Machine Learning*, pp. 5132–5143, 2020.
- [27] S. Reddi, Z. B. Charles, M. Zaheer, Z. Garrett, K. Rush, J. Konečný, S. Kumar, and B. McMahan, “Adaptive Federated Optimization,” in *Proceedings of International Conference on Learning Representations*, 2021.
- [28] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, “Federated Optimization in Heterogeneous Networks,” in *Proceedings of Machine Learning and Systems*, pp. 429–450, 2020.
- [29] X. Wu, F. Huang, Z. Hu, and H. Huang, “Faster Adaptive Federated Learning,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 10379–10387, 2023.
- [30] L. Gao, H. Fu, L. Li, Y. Chen, M. Xu, and C.-Z. Xu, “FedDC: Federated Learning With Non-IID Data via Local Drift Decoupling and Correction,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 10112–10121, 2022.
- [31] X. Wu, H. Huang, Y. Ding, H. Wang, Y. Wang, and Q. Xu, “FedNP: Towards Non-IID Federated Learning via Federated Neural Propagation,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 10399–10407, 2023.
- [32] L. Liu, X. Jiang, F. Zheng, H. Chen, G.-J. Qi, H. Huang, and L. Shao, “A Bayesian Federated Learning Framework With Online Laplace Approximation,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 1, pp. 1–16, 2024.
- [33] M. Al-Shedivat, J. Gillenwater, E. Xing, and A. Rostamizadeh, “Federated Learning via Posterior Averaging: A New Perspective and Practical Algorithms,” in *Proceedings of International Conference on Learning Representations*, 2021.

- [34] S. Jiang, F. Firouzi, and K. Chakrabarty, “Low-Overhead Clustered Federated Learning for Personalized Stress Monitoring,” *IEEE Internet of Things Journal*, vol. 11, no. 3, pp. 4335–4347, 2024.
- [35] E. Yu, Z. Ye, Z. Zhang, L. Qian, and M. Xie, “A Federated Recommendation Algorithm Based on User Clustering and Meta-Learning,” *Applied Soft Computing*, vol. 158, p. 111483, 2024.
- [36] Y. Ruan and C. Joe-Wong, “FedSoft: Soft Clustered Federated Learning with Proximal Local Updating,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 8124–8131, 2022.
- [37] A. Ghosh, J. Hong, D. Yin, and K. Ramchandran, “Robust Federated Learning in a Heterogeneous Environment,” *arXiv preprint arXiv:1906.06629*, 2019.
- [38] C. Briggs, Z. Fan, and P. Andras, “Federated Learning with Hierarchical Clustering of Local Updates to Improve Training on Non-IID Data,” in *Proceedings of International Joint Conference on Neural Networks*, pp. 1–9, 2020.
- [39] L. Huang, A. L. Shea, H. Qian, A. Masurkar, H. Deng, and D. Liu, “Patient Clustering Improves Efficiency of Federated Machine Learning to Predict Mortality and Hospital Stay Time Using Distributed Electronic Medical Records,” *Journal of Biomedical Informatics*, vol. 99, p. 103291, 2019.
- [40] “Chicago public crime data - 2019.” https://data.cityofchicago.org/Public-Safety/Crimes-2019/w98m-zvie/about_data. Accessed: 2023-09-30.
- [41] D. Yang, D. Zhang, and B. Qu, “Participatory Cultural Mapping Based on Collective Behavior Data in Location-Based Social Networks,” *ACM Trans. Intell. Syst. Technol.*, vol. 7, no. 3, pp. 30:1–30:23, 2016.